Knowledge Representation

CS3263

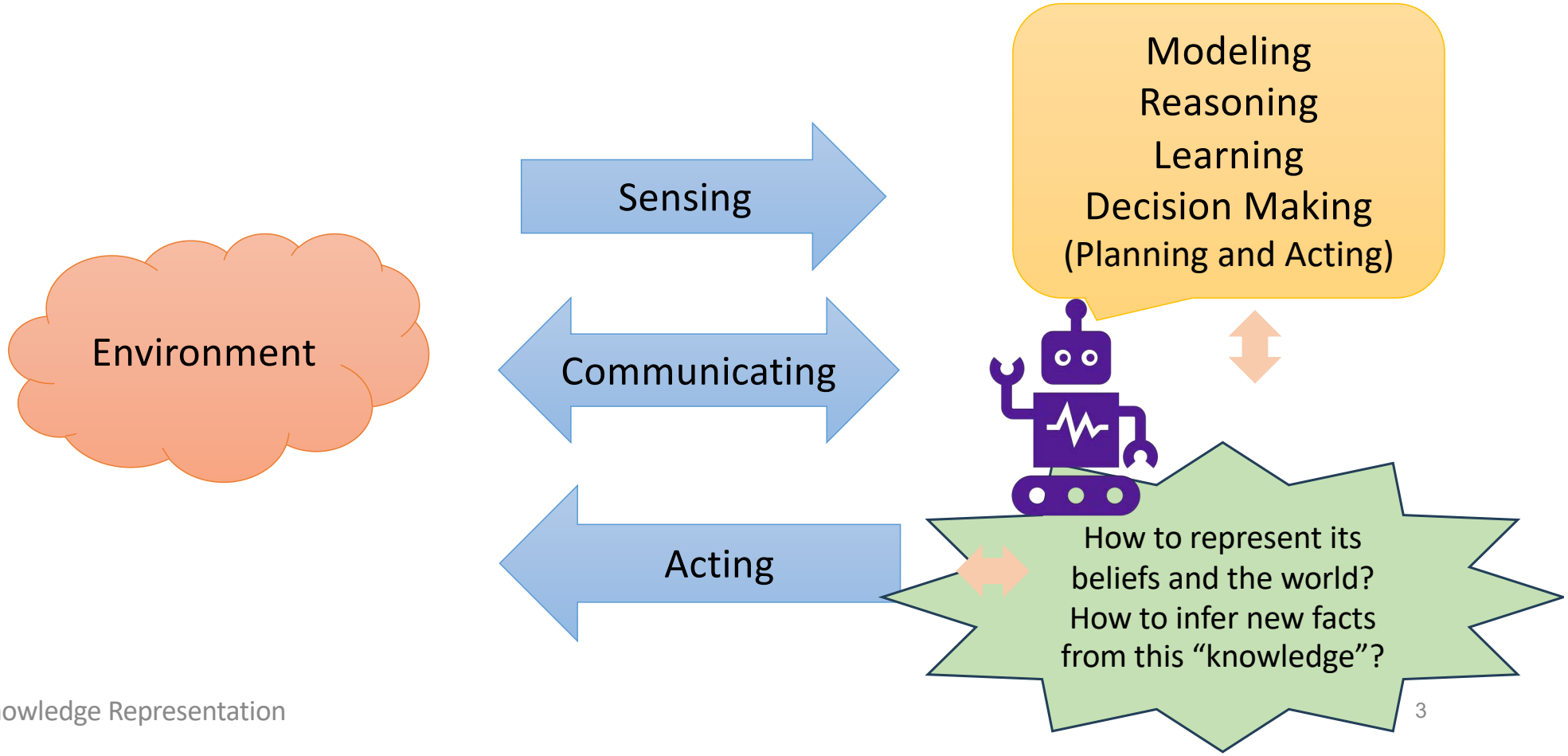
Foundations of Artificial Intelligence

Version 4.0

Topics

- Fundamentals of Knowledge Based AI Systems
  - Knowledge Based Reasoning
  - Knowledge Representation Spectrum
- Logic and Inference
  - Model Checking
  - Theorem Proving
    - Forward Chaining
    - Backward Chaining
    - *Resolution*
- Logic-Based Representation and Reasoning Systems

AI: Building A Rational Agent

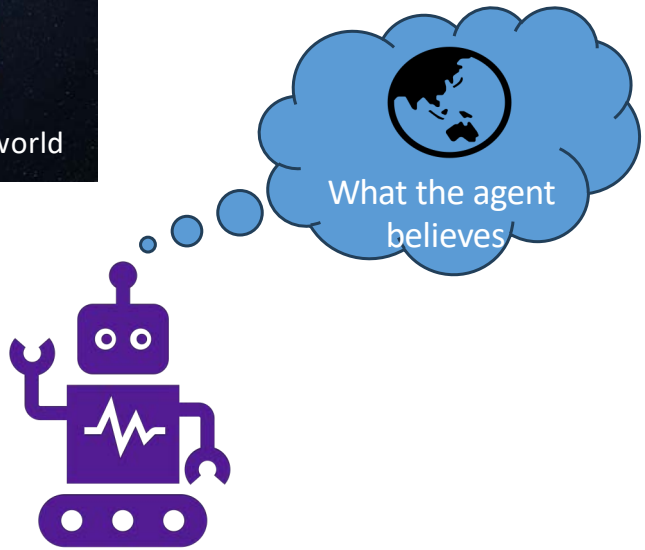


Knowledge Based Systems

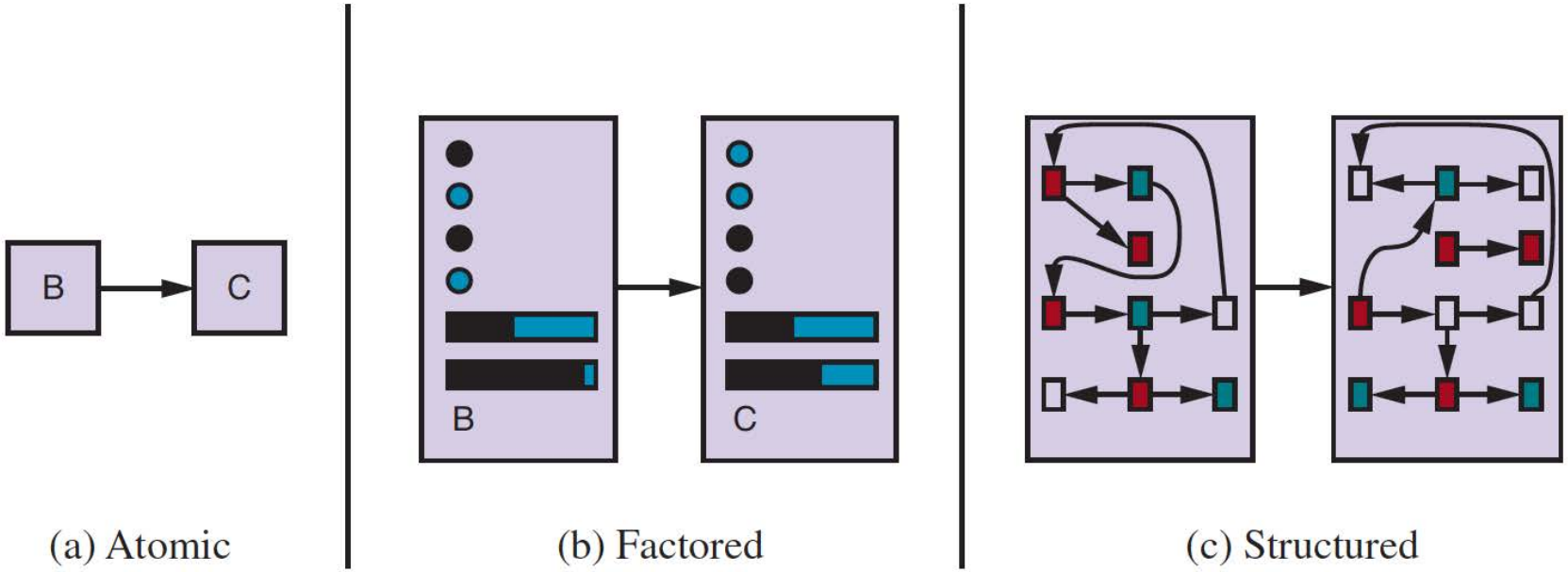
Declarative approach to representation and reasoning

Knowledge Based Reasoning

- An AI agent must:
  - Represent the world
  - Reason about that representation
  - Act based on inferred consequences
- Knowledge-based AI separates:
  - What is known (representation)
  - How conclusions are derived (inference)
- Central question:
  - What representations support reliable inference?



Knowledge Representation Spectrum



(a) Atomic

(b) Factored

(c) Structured

Search, game-playing

CSPs, planning, propositional logic, Bayesian networks, neural networks

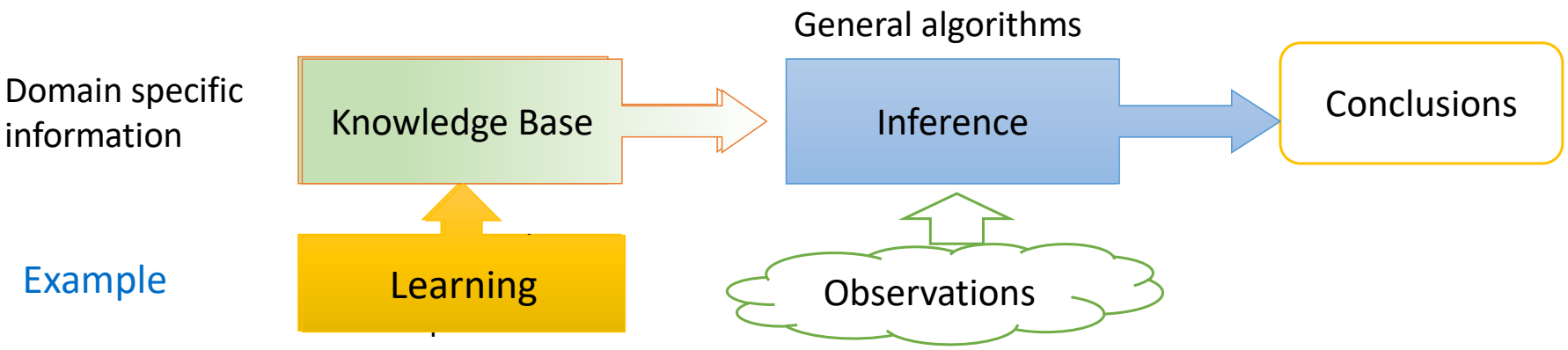
First order logic, databases, probabilistic programs

Source: RN Figure 2.16

Notes: Knowledge Representation Spectrum

- Different representations support different kinds of reasoning:
  - Search, CSPs → operational reasoning
  - Logic → sound, symbolic inference
  - Probabilistic models → uncertainty
  - Neural models → pattern-based generalization
- Key takeaway:
  - Logic is one point in the spectrum: exact, interpretable, but limited.

Knowledge Based Systems



Logic-Based Reasoning: Guarantees vs Flexibility

- Logic-based representations provide:
  - Precise semantics
  - Truth-preserving inference
  - Verifiable conclusions
- Trade-offs:
  - Expressiveness vs tractability
  - Certainty vs uncertainty
- Assumption:
  - If the knowledge base is true, inferred conclusions are true

Roles of Logic in Modern AI Systems

- Where logic is used today:
  - Tool and action planning
  - Safety and policy enforcement
  - Knowledge graph reasoning
  - Verification of large foundation model (LFM) outputs
- Note:
  - Logic is no longer just the “brain” (e.g., planning, constraints, compliance)
  - Logic is the governor and verifier
- Key message:
  - LFMs generate. Logic guarantees.

Fundamentals of Logic

Introduction to logical reasoning

Logic

- Formal language
  - A set of expressions or sentences obtained using an alphabet (of symbols) and rules
- Syntax:
  - Which sentences are allowed (well-formed)?
- Semantics:
  - Model definition: What are the possible worlds?
  - Truth definition: Which sentences are true in which models (possible worlds)?
- Inference:
  - Is a sentence true in a world where another sentence (or set of sentences) is true?

Examples: Logic systems

- Propositional Logic – declarative and compositional
  - Syntax: $P \wedge (Q \vee \neg R)$; $A1 \vee A2 \Leftrightarrow (B \Longrightarrow C)$
  - Interpretation (Model):
    - Assign truth values to propositional variables: $\{P = True,\ Q = False,\ R = True\}$
  - Semantics: A sentence $(\alpha \Longrightarrow \beta)$ is false only if α is true and β is false.
  - Scope: Focuses on propositions that are either true or false
- First-order Logic (FOL) – alco called Predicate Logic
  - Syntax: $\forall x\, \exists y\, P(x,y) \wedge \neg Q(y, f(x), Apple) \Rightarrow f(x) \wedge f(y)$
  - Interpretation (Model):
    - Domain : $\{\tau_1, \tau_2, \tau_3\}$
    - Relations: P holds for $\langle \tau_1, \tau_2 \rangle$; Q holds for $\langle \tau_1, \tau_2\ \tau_3 \rangle$.
    - Functions: $f(\tau_2) = \tau_3$; $Apple = \tau_3$.
    - Variables map to domain elements under this interpretation.
  - Semantics: A sentence $\phi(\sigma)$ is true in a world if σ is assigned to some τ_i in the domain and ϕ holds for τ_i
  - Scope: Supports reasoning over sets of objects and their properties

Other logics:
Modal logic
Temporal logic
…

the interpretation maps σ to τ_i

Examples: Logical Inference

- Propositional logical inference
  - Socrates is a man.
  - If Socrates is a man,
    then Socrates is mortal.
  - Therefore, Socrates is mortal.
- First order logical inference
  - All humans are mortal.
  - Socrates is a human.
  - Therefore, Socrates is mortal.

- What about:
  - All fish are mortal.
  - I am mortal.
- …

Argument:

Conclusion

Satisfiability

- If a sentence α is true in model m:
  - m satisfies α
  - m is a model of α
- Let $M(\alpha)$ be the set of all models of α

x = 2, y = 2 satisfies "x + y = 4"
x = 2, y = 2 is a model of "x + y = 4"

M("x + y = 4") = {(x = 2, y = 2), (x=3,y=1), …}

- A sentence α is ***satisfiable*** if there exists a possible world or model (truth-value assignment) under which α evaluates to $True$.

Satisfiability

- Example: Propositional logic
  - Sentence α: $P \ \wedge \ (Q \ \vee \ \neg R)$
    - Possible world or model M_1: $\{P = True, \ Q = False, \ R = True\}$
    - Possible world or model M_2: $\{P = True, \ Q = False, \ R = False\}$

Is M_1 a model of α?

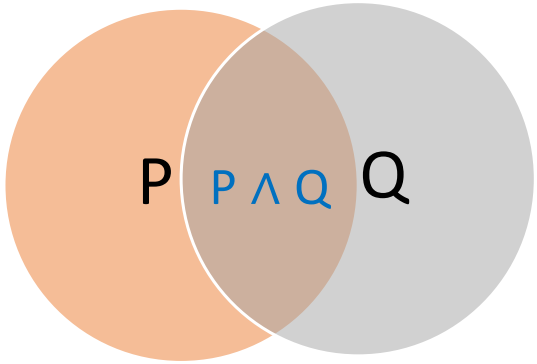
- Example: First order logic
  - Sentence ϕ: $\exists x(R(x) \wedge \neg S(x))$
  - Consider domain of a set of animals: $R(x)$ denotes "x is a bird", and $S(x)$ denotes "x can swim".

    Possible world or model I_1: $\{x: \ A\ group\ of\ penguins\ in\ Antarctica\}$

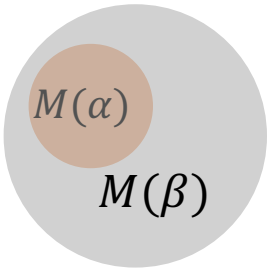
    Possible world or model I_2: $\{x: \ A\ nest\ of\ sparrows\ on\ a\ tree\ outside\ your\ apartment\}$.
  - Note: If there exists a possible world or model where there is at least one bird that cannot swim, then ϕ is satisfiable.

Is I_1 a model of ϕ ?

Entailment



- α entails β
- $\alpha \vDash \beta$ iff in every model in which α is true, β is also true
- $\alpha \vDash \beta$ iff $M(\alpha) \subseteq M(\beta)$



- Note:
  - α is a stronger assertion than β: it rules out more possible worlds
  - E.g., in propositional logic, where P and Q are propositions:

    $P \wedge Q$ entails P: $\quad M(P \wedge Q) \subseteq M(P)$

Validity

- A sentence α is valid if it is true in all models.
  - E.g., the sentence $P \lor \neg P$ is valid.
  - E.g., the sentence $\forall x\ (human(x) \lor \neg human(x))$ is valid.
- A valid sentence α is called a tautology — it is necessarily true.
  - Since the sentence $True$ is true in all models, every sentence that is valid is logically equivalent to $True$.
- A sentence α is called a contradiction if it is false in all models.
  - E.g., the sentence $P \land \neg P$ is a contradiction.
  - E.g., the sentence $\exists x\ (human(x) \land \neg human(x))$ is a contradiction.

Summary: Satisfiability, Validity, Entailment

- Validity and satisfiability are of course connected:
  - α is valid iff $\neg\alpha$ is unsatisfiable;
  - Contrapositively, α is satisfiable iff $\neg\alpha$ is not valid.
- Checking entailment with satisfiability
  - $\alpha \vDash \beta$ if and only if the sentence $\alpha \wedge \neg\beta$ is unsatisfiable.
- Note:
  - Question of entailment for first-order logic is semidecidable
    - Algorithms exist that say yes to every entailed sentence
    - No algorithm exists that also says no to every non-entailed sentence.

Logical Inference

- To show entailment ($\alpha \vDash \beta$):
  - α entails β if, in every model where α is true, β is also true.
    i.e., $M(\alpha) \subseteq M(\beta)$, where $M(x)$ is the set of all models that satisfy x
- Model Checking:
  - Enumerates all models to confirm β is true in all models where α is true.
- Theorem Proving:
  - Searches for proofs (valid derivations) of β from α using inference rules, without enumerating models.
  - Often more efficient when the number of models $|M|$ is large, but a proof (if it exists) is relatively short.

Logical Inference in KBS

- Knowledge Base and Entailment (α in $\alpha \vDash \beta$ is replaced by a Knowledge Base KB)
  - If KB is true in real world, then any sentence ϕ soundly derived from KB must also be true.
  - To show entailment $KB \vDash \phi$, we can either:

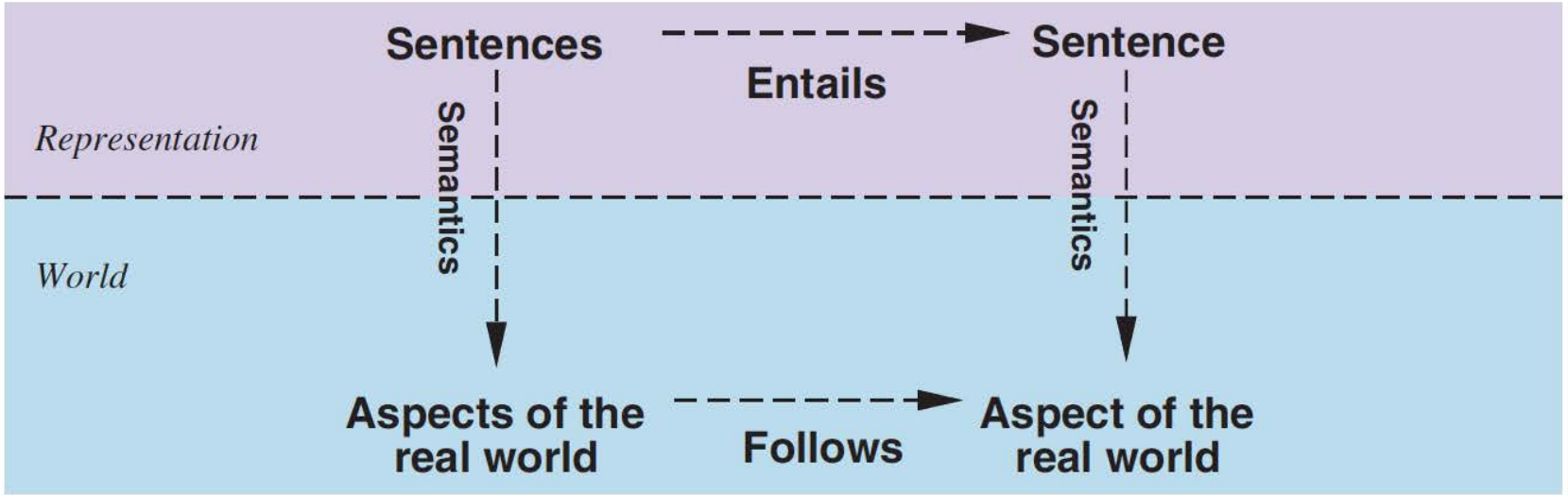
1. Model Check all models of KB to confirm ϕ also holds in each.
2. Prove ϕ via sound inference algorithm i, written as $KB \vdash_i \phi$ to indicate i derives ϕ from KB

- Sound and Complete Inference
  - An inference algorithm i is sound (truth-preserving) if whenever $KB \vdash_i \phi$, then $KB \vDash \phi$.
  - It is complete if whenever $KB \vDash \phi$, then $KB \vdash_i \phi$.
  - A sound and complete inference procedure correctly answers any query entailed by KB.

Monotonicity

- Set of entailed sentences can only increase as information is added to the knowledge base
- For a set of sentences KB and any sentences α and β:
  - If $KB \vDash \alpha$ then $KB \wedge \beta \vDash \alpha$
- Implications:
  - If KB is true in the real world, then any sentence α derived from KB by a sound inference procedure is also true in the real world
  - Inference rule can be applied with suitable premises found in the KB – conclusion of the rule must follow regardless of what else is in the KB

Logical Representation and Reasoning



Source: RN Figure 7.6

Embarking on Martian Adventures …

- Persy, the Mars Rover, explores Mars to detect and analyze interesting artifacts.
- It receives signals from different zones, which may suggest the presence of new life forms.
- Zones with signals require inspection, and adjacent zones should be noticed for future exploration.

Generated by Dalle-3 Beta

Model Checking

Enumerating models

What is a Model (in Logic)?

- Propositional logic:
  - An assignment of true or false to every propositional symbol
  - E.g., For 3 propositional symbols – P, Q, R, there will be 2^3 possible models—one for each combination of True or False.
- First Order Logic (FOL):
  - A domain of discourse and an interpretation function provide a "realization" of all non-logical symbols (constants, predicates, functions), thereby determining the truth values of sentences.

Model Checking

- Input: A knowledge base KB and a formula α.
- Goal: To show entailment $KB \vDash \alpha$.
- Procedure (Enumeration Method):
  1. Enumerate all possible interpretations (or "worlds").
  2. Identify those interpretations M in which KB is true (i.e., $M \vDash KB$).
  3. Check whether α is also true in each such M.
  - If α holds in every model of KB, then $KB \vDash \alpha$. Equivalently, we verify that

$$M(KB) \subseteq M(\alpha)$$

where $M(\phi)$ denotes the set of all models of a formula (or set of formulas) ϕ.

Example: Martian Adventures 1

- Story:
  - Persy the Mars Rover is tasked with exploring the planet and collecting samples.
  - Everyday, Persy must follow some simple rules each day to ensure he is operating safely and efficiently. Before exploring, he checks if:
    - His solar panels are fully charged.
    - There is no dust storm.
    - His radar navigation system that works in dust storms is working
- Propositions:
  - P: Persy's solar panels are fully charged.
  - Q: There is a dust storm on Mars.
  - R: Radar navigation system working.
  - S : It is a safe day to explore

Example: Model Checking

P: Persy's solar panels are fully charged.
Q: There is a dust storm on Mars.
R: Radar navigation system working.
S : It is a safe day to explore

- Safety protocol is represented by the formula:

$$P \wedge \big(R \vee (\neg R \wedge \neg Q)\big) \Rightarrow S$$

- One day, the following status update is received from Persy:
  - $P = True$ (panels charged)
  - $Q = False$ (no dust storm)
  - $R = False$ (radar not working)
- Knowledge Base: (all facts + rules)
  - $KB = \{P, \neg Q, \neg R, P \wedge (R \vee (\neg R \wedge \neg Q)) \Rightarrow S\}$
- Query: Is it safe to explore? (Show $KB \vDash S$, i.e., S holds in all models of the KB.
- Model checking
  - Compute $P \wedge (R \vee (\neg R \wedge \neg Q))$: $True \wedge (False \vee (True \wedge True)) = True$
  - Since the antecedent is true in all models of KB, the implication can hold only if S is true.
- Conclusion: $KB \vDash S$
  - **Based on the protocol and today's observations, Persy can safely operate.**

Notes

We list all $2^4 = 16$ valuations for (P, Q, R, S).

P	Q	R	S	$\neg R$	$\neg Q$	$(\neg R \wedge \neg Q)$	$R \vee (\neg R \wedge \neg Q)$	$P \wedge \ldots$	F
T	T	T	T	F	F	F	T	T ∧ T = T	**T ⇒ T = T**
T	T	T	F	F	F	F	T	T ∧ T = T	**T ⇒ F = F**
T	T	F	T	T	F	F	F	T ∧ F = F	**F ⇒ T = T**
T	T	F	F	T	F	F	F	T ∧ F = F	**F ⇒ F = T**
T	F	T	T	F	T	F	T	T ∧ T = T	**T ⇒ T = T**
T	F	T	F	F	T	F	T	T ∧ T = T	**T ⇒ F = F**
T	F	F	T	T	T	T	T	T ∧ T = T	**T ⇒ T = T**
T	F	F	F	T	T	T	T	T ∧ T = T	**T ⇒ F = F**
F	T	T	T	F	F	F	T	F ∧ T = F	**F ⇒ T = T**
F	T	T	F	F	F	F	T	F ∧ T = F	**F ⇒ F = T**
F	T	F	T	T	F	F	F	F ∧ F = F	**F ⇒ T = T**
F	T	F	F	T	F	F	F	F ∧ F = F	**F ⇒ F = T**
F	F	T	T	F	T	F	T	F ∧ T = F	**F ⇒ T = T**
F	F	T	F	F	T	F	T	F ∧ T = F	**F ⇒ F = T**
F	F	F	T	T	T	T	T	F ∧ T = F	**F ⇒ T = T**
F	F	F	F	T	T	T	T	F ∧ T = F	**F ⇒ F = T**

Model Checking Algorithms

- Checking entailment with satisfiability (SAT)
  - $\alpha \vDash \beta$ if and only if the sentence $\alpha \ \wedge \ \neg\beta$ is unsatisfiable.
  - $\alpha \vDash \beta$ iff $\alpha \ \Rightarrow \ \beta$ iff $\neg\alpha \vee \beta$ iff $(\alpha \ \wedge \ \neg\beta)$ is always false (unsatisfiable).

Sf. proof by contradiction or reductio ad absurdum in mathematics:
Assume β is false and show that this leads to contradiction with axiom α.

- Popular SAT algorithms: (for propositional logic)
  - Davis-Putnam Algorithm
    - Version by Davis, Logemann, and Loveland (1962) - DPLL
    - Intelligent backtracking + random restarts
  - WalkSat
    - Randomized local search
    - Useful when we expect solution to exist. Cannot always detect unsatisfiability.

SAT solvers usually work with formulas in conjunctive normal form (CNF) - a conjunction (AND) of a set of clauses (OR of literals). Each clause can be considered a constraint and all constraints must be satisfied.

- Note:
  - SAT is essentially a constraint satisfaction problem, so CSP-solving techniques apply.
  - SAT is a scalable alternative to truth-table–based model checking

Theorem Proving

Searching for proofs
Deriving conclusions without enumerating models

Theorem Proving

A chain of conclusions that leads to the desired goal

- Theorem proving
  - To show entailment $KB \vDash \alpha$
  - Check if an inference algorithm i can derive α from KB:

    $KB \vdash_i \alpha$ (α is derived from KB by i OR i derives α from KB)
  - Method: Apply inference rules to sentences in KB to construct a proof leading to α
- What are the inference rules?
  - Valid inference rules in derivation system
  - Forward chaining
  - Backward chaining
  - Resolution
- Question: How to use inference rules to derive a proof?
  - Are they all sound and complete?

The Deduction Theorem

- The Deduction Theorem (From definition of entailment):
  - For any sentences α and β:
    $\alpha \models \beta$ if and only if the sentence $(\alpha \Rightarrow \beta)$ is valid
  - Decide if $\alpha \models \beta$:
    a) by checking that $(\alpha \Rightarrow \beta)$ is true in every model or
    b) by proving that $(\alpha \Rightarrow \beta)$ is equivalent to True
- Conversely, the deduction theorem states that:
  - Every valid implication sentence describes a legitimate inference.
- Recall: Checking entailment with satisfiability
  - $\alpha \models \beta$ if and only if the sentence $(\alpha \wedge \neg\beta)$ is unsatisfiable

Logical Arguments

Socrates is a man.
If Socrates is a man,
then Socrates is mortal.
Therefore, Socrates is mortal

- A logical argument:
  - Consists of a set of sentences, $A_1, A_2, \ldots, A_n$, called the premises, and a sentence B, called the conclusion.
  - The premises entail the conclusion if in every model in which all the premises are true, the conclusion is also true.
  - If the premises entail the conclusion, the argument is sound

$$A_1, A_2, \ldots, A_n \vDash B$$

  - If the premises do not entail the conclusion, the argument is a fallacy.

$$A_1, A_2, \ldots, A_n \nvDash B$$

  - An argument without premises is simply a tautology.

Source: NJ 2.1

Inference and Proofs

- Inference rules can be applied to derive a proof.
- Best known rule is Modus Ponens

$$\frac{\alpha \Rightarrow \beta, \alpha}{\beta}$$

Notation means that whenever sentences $\alpha \Rightarrow \beta$ and α are given, then sentence β can be inferred.

- Can also use logical equivalences (e.g. on the right) as inference rules.

$$
\begin{array}{rcll}
(\alpha \wedge \beta) & \equiv & (\beta \wedge \alpha) & \text{commutativity of } \wedge \\
(\alpha \vee \beta) & \equiv & (\beta \vee \alpha) & \text{commutativity of } \vee \\
((\alpha \wedge \beta) \wedge \gamma) & \equiv & (\alpha \wedge (\beta \wedge \gamma)) & \text{associativity of } \wedge \\
((\alpha \vee \beta) \vee \gamma) & \equiv & (\alpha \vee (\beta \vee \gamma)) & \text{associativity of } \vee \\
\neg(\neg\alpha) & \equiv & \alpha & \text{double-negation elimination} \\
(\alpha \Rightarrow \beta) & \equiv & (\neg\beta \Rightarrow \neg\alpha) & \text{contraposition} \\
(\alpha \Rightarrow \beta) & \equiv & (\neg\alpha \vee \beta) & \text{implication elimination} \\
(\alpha \Leftrightarrow \beta) & \equiv & ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)) & \text{biconditional elimination} \\
\neg(\alpha \wedge \beta) & \equiv & (\neg\alpha \vee \neg\beta) & \text{De Morgan} \\
\neg(\alpha \vee \beta) & \equiv & (\neg\alpha \wedge \neg\beta) & \text{De Morgan} \\
(\alpha \wedge (\beta \vee \gamma)) & \equiv & ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma)) & \text{distributivity of } \wedge \text{ over } \vee \\
(\alpha \vee (\beta \wedge \gamma)) & \equiv & ((\alpha \vee \beta) \wedge (\alpha \vee \gamma)) & \text{distributivity of } \vee \text{ over } \wedge
\end{array}
$$

Notes: Deduction System

- A set of inference rules is called a deduction system.
  - A deduction system is sound if it only derives sound arguments.
  - A deduction system is complete if it can derive every sound argument.
- Is this set of inference rules sound and complete?
  - YES for propositional logic

Inference Rule	Name	
$A, B \vDash A \wedge B$	Combination rule	CR
$A \wedge B \vDash A$	Simplification rule	SR
$A \vDash A \vee B$	Addition rule	AR
$A, A \Rightarrow B \vDash B$	Modus ponens	MP
$\neg B, A \Rightarrow B \vDash \neg A$	Modus tolens	MT
$A \Rightarrow B, B \Rightarrow C \vDash A \Rightarrow C$	Hypothetical syllogism	HS
$A \vee B, \neg A \vDash B$	Disjunctive syllogism	DS
$A \Rightarrow B, \neg A \Rightarrow B \vDash B$	Rule of cases	RC
$A \Leftrightarrow B \vDash A \Rightarrow B$	Equivalence elimination	EE
$A \Rightarrow B, B \Rightarrow A \vDash A \Leftrightarrow B$	Equivalence introduction	EI
$A, \neg A \vDash B$	Inconsistency rule	IR
$A \wedge B \vDash B \wedge A$	"and" Commutivity rule	ACR
$A \vee B \vDash B \vee A$	"or" Commutivity rule	OCR
If $A_1, A_2, \ldots, A_n, B \vDash C$ then $A_1, A_2, \ldots, A_n \vDash B \Rightarrow C$	Deduction theorem	DT

Source: NJ Table 2.2

Notes: Complete Deduction Systems

- Modus Ponens by itself is incomplete
  - From $Rain$, $Rain \vee Sprinkler \Rightarrow Wet$, Modus Ponens (MP) cannot infer Wet because $Rain \vee Sprinkler$ is not available
  - With $\vee$ -introduction/addition: $\alpha/(\alpha \vee \beta)$, we can derive $Rain \vee Sprinkler$ then infer Wet by MP
- Gödel's Completeness Theorem (about FOL as a logic):
  - There exist sound and complete proof systems for first-order logic: if $KB \vDash \alpha$, then $KB \vdash_i \alpha$
  - FOL has sound and complete deduction systems: every logically entailed sentence is provable.
- Church–Turing Undecidability (about algorithms):
  - First-order entailment is undecidable: there is no algorithm that always halts and correctly decides entailment for all first-order sentences (it is only semi-decidable).

Example: Chatty GoPTy

- Problem: Determine how Chatty GoPTy earns a living.
  - We know that Chatty GoPTy either writes essay or helps other people to write essays.
  - We also know that if Chatty GoPTy helps other people to write essays, then she earns her living as an English professor.
  - Finally, we know that Chatty GoPTy does not write essays.
- Let:
  - P: Chatty GoPTy writes essays
  - Q: Chatty GoPTy helps other people to write essays
  - R: Chatty GoPTy is an English professor

Example: Chatty GoPTy (cont.)

- Let:
  - P: Chatty GoPTy writes essays
  - Q: Chatty GoPTy helps other people to write essays
  - R: Chatty GoPTy is an English professor

1.	$P \vee Q$	Premise	
2.	$Q \Rightarrow R$	Premise	
3.	$\neg P$	Premise	
4.	?	1, 3, DS	$A \vee B, \neg A \vDash B$
5.	?	2, 4, MP	$A, A \Rightarrow B \vDash B$

Forward Chaining and Backward Chaining

Forward vs Backward Chaining

- Forward chaining (data-driven):
  - Start from known facts
  - Apply rules to derive new facts
  - Useful when many conclusions are needed
- Backward chaining (goal-driven):
  - Start from a query (goal)
  - Work backward to required facts
  - Useful when answering specific queries
- Key takeaway:
  - Same inference rules, different control strategy.
  - Both are sound; completeness depends on clause restrictions

Unification in FOL

- A unification of two sentences A and B: a substitution θ (called unifier) of values for some variables in A and B that make the sentences identical.
- Example:
  - Unify: $loves(Dave, y)$ and $loves(x, Gloria)$.
  - Unify: $parents(Dave, y, z)$ and $parents(y,\ Mary, Sam)$.
  - Unify: $parents(Dave, Nancy, z)$ and $parents(y,\ Mary, Sam)$
  - Unify: $parents(Dave, x, Sam)$ and $parents(x, Mary, Sam)$.

Source: NJ 3.1.5

Most General Unifier

- A unifier θ is a most general unifier (MGU) if every other unifier θ' is an instance of θ: θ' can be derived by making substitutions in θ.
- Example:
  - Unify: $father(x, Sam)$ and $father(y, z)$
  - $\theta_1 = \{x/Dave, y/Dave, z/Sam\}$ unifies to: $father(Dave, Sam)$.
  - $\theta_2 = \{x/y, z/Sam\}$ unifies to: $father(y, Sam)$
  - θ_2 is more general than θ_1: $father(Dave, Sam)$ is an instance of $father(y, Sam)$.
  - Unifier θ_2 is a most general unifier.
- Note:
  - A variable cannot be replaced by a term containing the variable being unified
  - Occur check: substitution $x/f(x)$ is NOT allowed.

Source: NJ 3.1.5

Forward Chaining

Source: Wikipedia

Type of Horn clause	Disjunction form	Implication form	Read intuitively as
Definite clause	$\neg p \vee \neg q \vee \ldots \vee \neg t \vee u$	$u \leftarrow p \wedge q \wedge \ldots \wedge t$	assume that, if *p* and *q* and ... and *t* all hold, then also *u* holds
Fact	u	$u \leftarrow true$	assume that *u* holds
Goal clause	$\neg p \vee \neg q \vee \ldots \vee \neg t$	$false \leftarrow p \wedge q \wedge \ldots \wedge t$	show that *p* and *q* and ... and *t* all hold[5]

- Algorithm:
  - To prove: $A_1, A_2, \ldots, A_n$, $A_1 \wedge A_2 \wedge \cdots \wedge A_n \Rightarrow B \vDash B$ (Modus Ponens!)
  - Start from facts
  - Apply Modus Ponens to fire rules whose premises are satisfied
  - Add new facts until saturation or query q: B derived

Notes: Forward Chaining Algorithm

continually infers new facts until no new facts can be inferred

For each rule in the KB, try to find a match for the antecedent of the rule from what's currently known in the KB.

```
function FOL-FC-ASK(KB, α) returns a substitution or false
  inputs: KB, the knowledge base, a set of first-order definite clauses
          α, the query, an atomic sentence

  while true do
     new ← { }        // The set of new sentences inferred on each iteration
     for each rule in KB do
        (p1 ∧ ... ∧ pn ⇒ q) ← STANDARDIZE-VARIABLES(rule)
        for each θ such that SUBST(θ, p1 ∧ ... ∧ pn) = SUBST(θ, p'1 ∧ ... ∧ p'n)
                 for some p'1, ..., p'n in KB
           q' ← SUBST(θ, q)
           if q' does not unify with some sentence already in KB or new then
              add q' to new
              φ ← UNIFY(q', α)
              if φ is not failure then return φ
     if new = { } then return false
     add new to KB
```

If a match is found, the consequent of the rule is inferred (and possibly with some variables substituted based on the match).

If the inferred fact matches the query, return the substitution

If no new facts can be inferred in a full loop through the KB, return `false`, indicating that the query cannot be inferred from the KB.

Source: RN Figure 9.3

Notes: Explanations of FOL-FC-ASK

1. Continually infer new facts until no new facts can be inferred.
2. For each rule in the KB, try to find a match for the antecedent of the rule from what's currently known in the KB.
3. If a match is found, the consequent of the rule is inferred (and possibly with some variables substituted based on the match).
4. If the inferred fact matches the query, return the substitution.
5. If no new facts can be inferred in a full loop through the KB, return `false`, indicating that the query cannot be inferred from the KB.

- Sound: each step is generalized modus ponens.
- Complete for definite clause knowledge base.

Backward Chaining

- Backward chaining:
  - Works backward from query q or goal
  - Goal-directed reasoning (from goal to observations or facts)
  - Often efficient for single queries (goal-directed)
- Algorithm:
  - If goal matches a known fact → success
  - Otherwise choose a rule whose head matches goal
  - Recursively prove subgoals

Notes: Backward Chaining Algor

Check for all clauses that might unify with goal

Begin search with initial query and an empty substitution.

```
function FOL-BC-ASK(KB, query) returns a generator of substitutions
   return FOL-BC-OR(KB, query, { })

function FOL-BC-OR(KB, goal, θ) returns a substitution
   for each rule in FETCH-RULES-FOR-GOAL(KB, goal) do
      (lhs ⇒ rhs) ← STANDARDIZE-VARIABLES(rule)
      for each θ′ in FOL-BC-AND(KB, lhs, UNIFY(rhs, goal, θ)) do
         yield θ′

function FOL-BC-AND(KB, goals, θ) returns a substitution
   if θ = failure then return
   else if LENGTH(goals) = 0 then yield θ
   else
      first,rest ← FIRST(goals), REST(goals)
      for each θ′ in FOL-BC-OR(KB, SUBST(θ, first), θ) do
         for each θ″ in FOL-BC-AND(KB, rest, θ′) do
            yield θ″
```

Rule is standardized to ensure its variables don't conflict with other rules.

Try to prove all the goals in its list. If any of the goals can't be proven, the whole list fails.

If UNIFY(rhs,goal,$\boldsymbol{\theta}$) successful, solve for LHS conjuncts in FOL-BC-AND, else fail

Presumably retrieve all the rules from the KB where the consequent might match the current goal.

If there's no goal left, all goals have been proven; return current substitution

Separate first of remaining goals and prove with FOL-BC-OR. If succeeds, continue with rest of remaining goals

This version does not check for loops, so is not a complete algorithm

Source: RN Figure 9.6

Notes: Explanation of FOL-BC-ASK

- FOL-BC-ASK:
  - Initiates backward chaining with an initial query and empty substitution.
- FOL-BC-OR:
  - Examines all rules in the knowledge base to find matches for the goal.
  - Standardizes rule variables to avoid conflicts.
  - Unifies the goal with the rule's consequent and proceeds to prove the antecedent using FOL-BC-AND.
- FOL-BC-AND:
  - Tries to prove all goals in sequence.
  - Failure of any goal results in overall failure.
  - If all goals are proven, returns the substitution; otherwise, processes remaining goals recursively.
- Key Characteristics:
  - Follows an AND/OR search strategy.
  - Implemented as a generator using yield to provide multiple results.
  - Based on depth-first search, prone to repeated states and incompleteness.
  - Despite limitations, it is widely used in logic programming languages.

Example: Martian Adventures 3

- Scenario:
  - Persy must decide which zones to inspect based on geological signals
  - Signals: Water, Rock, Unknown Entity.
  - Zones: A, B, C, D, E.
- Forward Chaining Setting:
  - Objective: What are (all) the zones that Persy should inspect? (Start with Evidence)
- Backward Chaining Setting:
  - Objective: What are (all) the zones that Persy should notice? (Start with Goal)

Rules and Initial Information (State)

- Knowledge Base (Rules):

1. $\forall x(WaterSignal(x) \rightarrow Inspect(x))$
   - IF a zone has a water signal, THEN should be inspected.
2. $\forall x(RockSignal(x) \rightarrow Inspect(x))$
   - IF a zone has a rock signal, THEN it should be inspected.
3. $\forall x, y(Adjacent(x, y) \wedge Inspect(y) \rightarrow Notice(x))$
   - IF a zone is adjacent to an inspectable zone, THEN it should be noticed.

- Knowledge Base (Facts):
  - $WaterSignal(ZoneA)$
  - $UnknownEntity(ZoneB)$
  - $RockSignal(ZoneC)$
  - $Adjacent(ZoneD, ZoneC)$
  - $Adjacent(ZoneE, ZoneA)$

Forward Chaining

1. $\forall x(WaterSignal(x) \rightarrow Inspect(x))$
2. $\forall x(RockSignal(x) \rightarrow Inspect(x))$
3. $\forall x, y(Adjacent(x, y) \wedge Inspect(y) \rightarrow Notice(x))$

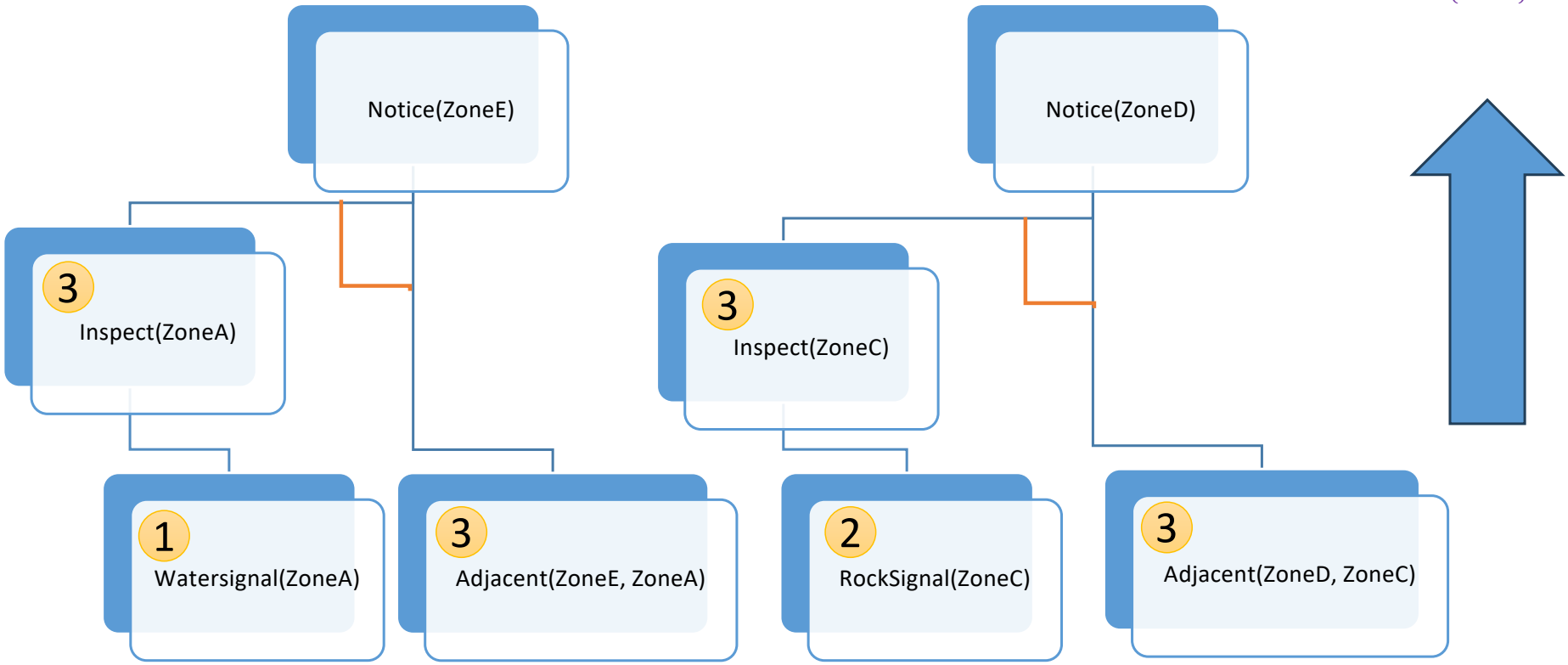
- $WaterSignal(ZoneA)$
- $UnknownEntity(ZoneB)$
- $RockSignal(ZoneC)$
- $Adjacent(ZoneD, ZoneC)$
- $Adjacent(ZoneE, ZoneA)$
- $Inspect(ZoneA)$
- $Inspect(ZoneC)$
- $Notice(ZoneE)$
- $Notice(ZoneD)$.

- Deduction from Water & Rock Signals
  1. Rule 1: From $WaterSignal(ZoneA)$, deduce $Inspect(ZoneA)$.
  2. Rule 2: From $RockSignal(ZoneC)$, deduce $Inspect(ZoneC)$.
- Evaluating Adjacent Zones
  3. Rule 3: Using $Inspect(ZoneA)$ and $Adjacent(ZoneE, ZoneA)$, deduce $Notice(ZoneE)$.
  4. Rule 3: Using $Inspect(ZoneC)$ and $Adjacent(ZoneD, ZoneC)$, deduce $Notice(ZoneD)$.

Proof Tree for Forward Chaining

1. $\forall x(WaterSignal(x) \rightarrow Inspect(x))$
2. $\forall x(RockSignal(x) \rightarrow Inspect(x))$
3. $\forall x, y(Adjacent(x, y) \wedge Inspect(y) \rightarrow Notice(x))$

- $WaterSignal(ZoneA)$
- $UnknownEntity(ZoneB)$
- $RockSignal(ZoneC)$
- $Adjacent(ZoneD, ZoneC)$
- $Adjacent(ZoneE, ZoneA)$
- $Inspect(ZoneA)$
- $Inspect(ZoneC)$
- $Notice(ZoneE)$
- $Notice(ZoneD)$.



Backward Chaining

1. $\forall x(WaterSignal(x) \rightarrow Inspect(x))$
2. $\forall x(RockSignal(x) \rightarrow Inspect(x))$
3. $\forall x, y(Adjacent(x, y) \wedge Inspect(y) \rightarrow Notice(x))$

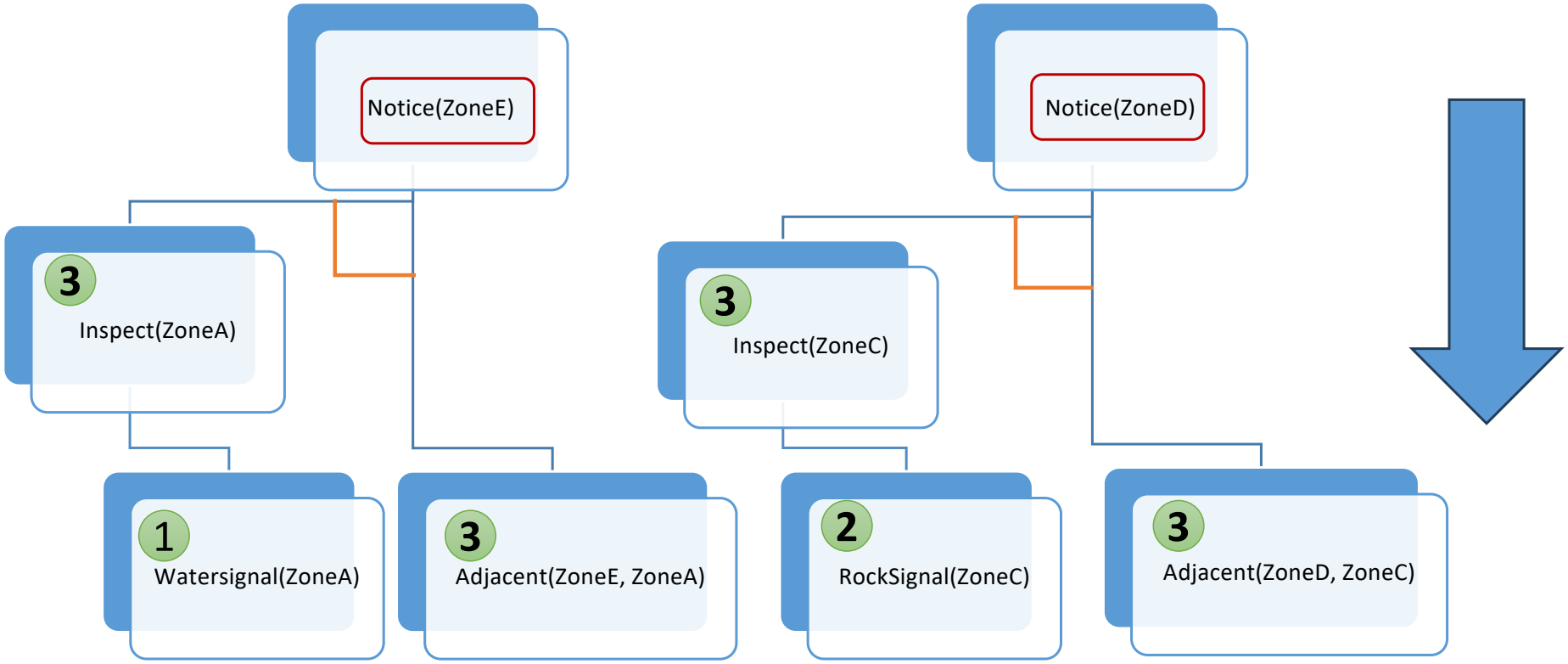
- $WaterSignal(ZoneA)$
- $UnknownEntity(ZoneB)$
- $RockSignal(ZoneC)$
- $Adjacent(ZoneD, ZoneC)$
- $Adjacent(ZoneE, ZoneA)$
- $Inspect(ZoneA)$
- $Notice(ZoneE)$
- $Inspect(ZoneC)$
- $Notice(ZoneD)$.

- From Hypothesis to Evidence
  1. Start with $Notice(ZoneE)$ hypothesis.
  2. Rule 3: Check rules to validate $Notice(ZoneE)$ – leads to $Inspect(ZoneA)$.
  3. Rule 1: Confrim $Inspect(ZoneA)$ using $WaterSignal(ZoneA)$.
  4. Check facts: confirm $Adjacent(ZoneE, ZoneA)$
  5. Rule 3: Confirm hypothesis $Notice(ZoneE)$
- From Hypothesis to Evidence
  1. Begin with $Notice(ZoneD)$ hypothesis.
  2. Rule 3: Check rules to validate $Notice(ZoneD)$ – leads to $Inspect(ZoneC)$.
  3. Rule 2: Confirm $Inspect(ZoneC)$ using $RockSignal(ZoneC)$.
  4. Check facts: – confirm $Adjacent(ZoneD, ZoneC)$
  5. Rule 3: Confirm hypothesis $Notice(ZoneD)$

Proof Tree for Backward Chaining

- $WaterSignal(ZoneA)$
- $UnknownEntity(ZoneB)$
- $RockSignal(ZoneC)$
- $Adjacent(ZoneD, ZoneC)$
- $Adjacent(ZoneE, ZoneA)$

1. $\forall x(WaterSignal(x) \rightarrow Inspect(x))$
2. $\forall x(RockSignal(x) \rightarrow Inspect(x))$
3. $\forall x, y(Adjacent(x, y) \wedge Inspect(y) \rightarrow Notice(x))$



Forward Chaining vs Backward Chaining

- Forward Chaining (FC):
  - FC provides a systematic way to deduce actions from data or evidence.
    - Zones for Inspection: A, C
    - Zones for Notice: D, E
  - Forward chaining is efficient when data continuously changes
- Backward Chaining (BC):
  - BC is valuable for verifying hypotheses.
    - Confirmed Zones for Inspection: A, C.
    - Validated Zones for Notice: D, E.
  - Backward chaining is efficient for evidence-based decision-making

Properties of Inference Algorithms

- Soundness
  - Forward and backward chaining are sound for:
  - Propositional logic with Horn clauses
  - First-order logic with definite clauses
  - Each step applies an if–then rule (Modus Ponens).
  - Every derived atom is entailed by the knowledge base.
- Completeness (restricted)
  - Forward and backward chaining are complete for atomic queries for:
  - Propositional Horn clauses
  - First-order definite clauses
  - Every entailed atomic fact will be derived.
  - Not complete for arbitrary formulas or full first-order logic.

Notes: Logic Programming (Operational View)

Definite clause: disjunction of literals where exactly one is positive

Horn clause: disjunction of literals where at most one is positive

- Main ideas:
  - Declarative: specify what should hold, not how to compute it.
  - Based on *definite clauses* - a subset of *Horn clauses* (rules and facts).
  - Inference is typically goal-driven (backward chaining).
  - Facts and rules form a logical database.
  - Reasoning applies if–then rules with variable matching.
  - Efficiency depends on rule structure; termination is not guaranteed.

Examples:
Prolog
Datalog

Notes: Logical Database Semantics (Assumptions)

- Assumptions in knowledge base and logic programming systems:
  - Unique Names Assumption (UNA):
    - Different constants denote different objects.
  - Closed World Assumption (CWA):
    - What cannot be derived is assumed false.
  - Domain Closure:
    - Only named objects exist in the domain.
  - These assumptions simplify inference and enable efficiency.
    - They are modeling choices, not universal truths.
    - Real-world alignment may be imperfect.

Logic-Based Representation and Reasoning Systems

Some major paradigms

Logic-Based Knowledge Representation

- Knowledge representation based on FOL
  - Many variations
  - Usually based on definite clauses
  - Restricted forms of first-order logic with specialized inference procedures
- Examples:
  - Production rules
  - Semantic networks
  - Knowledge graphs
  - Markov relational networks

Representation: Production Rules

- Production rules
  - A production rule is a two-part structure comprising an antecedent set of conditions and a consequent set of actions.
  - Common form: (as definite clauses)

IF Conditions THEN Actions

  - Antecedent conditions are tests to be applied to the current state of the KB
  - Consequent actions are a set of actions that modify the KB.
- Operations
  - Inference typically performed using forward chaining or backward chaining
  - Working of the production rules are usually kept in Working Memory part of KB

Ref: Brachman & Levesque, Chapter 7

Notes: Production Rule-Based System Operations

- Basic operation of a production system is a cycle of three steps that repeats until no more rules are applicable to the KB (WM):
  1. Recognize: find which rules are applicable, i.e., those rules whose antecedent conditions are satisfied by the current working memory;
  2. Resolve conflict: among the rules found in the first step (called a conflict set), choose which of the rules should "fire," or execute;
  3. Act: change the KB (WM) by performing the consequent actions of all the rules selected in the second step.
- Cycle repeats until no more rules can fire.
  - Note: Cycle operationalizes forward chaining over production rules

Ref: Brachman & Levesque, Chapter 7

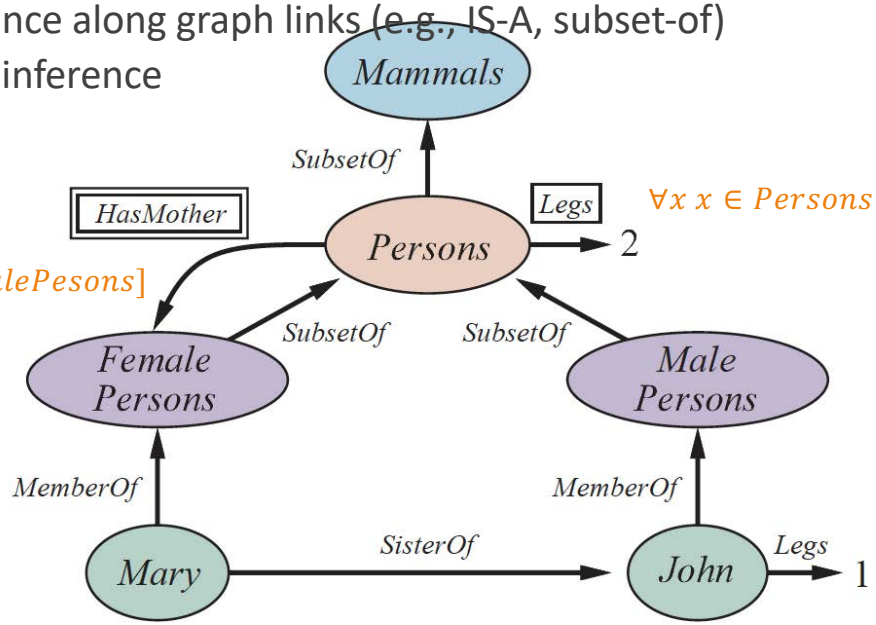
Representation: Semantic Networks

- Graph-based representation based on FOL
  - Many variations
  - Inference is typically based on inheritance along graph links (e.g., IS-A, subset-of)
  - Graph traversal corresponds to logical inference (e.g., inheritance = universal rules)

$\forall x\ x \in Persons \Rightarrow Legs(x, 2)$

$\forall x\ x \in Persons \Rightarrow [\forall y\ HasMother(x, y) \Rightarrow y \in FemalePesons]$

Inheritance

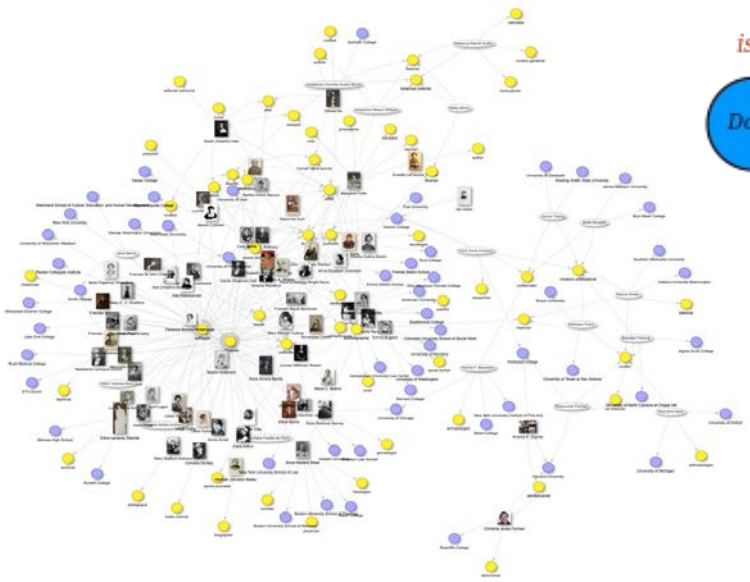


Source: RN Figure 10.4

Notes: Inference over Semantic Networks

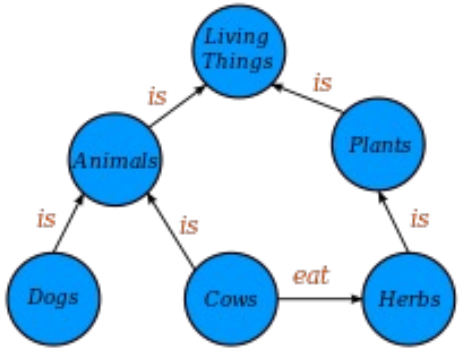
- Semantic networks represent knowledge as nodes and labeled links
  - (e.g., IS-A, subset-of, has-property).
- Inference is based on inheritance along graph links:
  - Properties of a class apply to its subclasses and instances.
- Example:
  - Fact: Mary IS-A Person
  - Rule: Person → has 2 legs
  - Inferred: Mary has 2 legs
- Operationally:
  - Inference corresponds to graph traversal along inheritance links.
- Logical view:
  - Inheritance encodes universal rules in first-order logic.

Representation: Knowledge Graphs

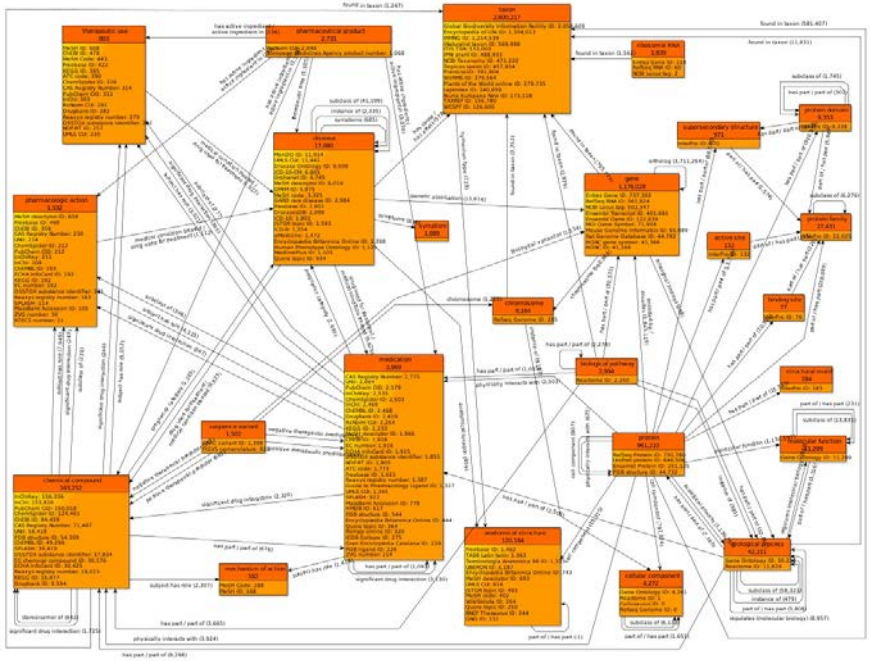


https://w.wiki/zmg

Fuzheado, CC BY-SA 4.0 <https://creativecommons.org/licenses/by-sa/4.0>, via Wikimedia Commons



Source: wikipedia



Andrew Su, CC BY 4.0 <https://creativecommons.org/licenses/by/4.0>, via Wikimedia Commons

Notes: Inference Over Knowledge Graphs

- Knowledge graphs
  - Represent facts as nodes and labeled edges (e.g., is-a, subset-of, has-property).
- Inference adds new facts by applying logical rules over the graph:
  - Inheritance (along is-a / subset-of links)
  - Rule application (IF–THEN rules over triples)
- Example:
  - Rule: $Person(x) \implies Mortal(x)$
  - Fact: $Person(Socrates)$
  - Inferred:$Mortal(Socrates)$
- Operationally:
  - Inference corresponds to forward chaining over graph edges.
- Key idea:
  - Knowledge graphs store facts; logic derives consequences.

Summary

- Knowledge-Based AI
  - Knowledge-based AI = knowledge + inference.
  - Reasoning is declarative.
- Logic as a Foundation
  - Logic is a formal representation language.
  - A logic has syntax, semantics, and inference rules.
- Logical Languages
  - Propositional logic reasons about sentence truth.
  - First-order logic adds objects, relations, and functions – combines formal and natural languages
- Logical Inference
  - Inference computes entailment via:
  - Model checking
  - Theorem proving

Homework

- Readings:
  - RN: 7.1– 7.5, *7.6 (Optional), 8.1 (Optional),* 8.2, 9.1- 9.4
  - NJ: 2.1, 3.1

- References:
  - (RN) Russell, S. and P. Norvig, Artificial intelligence: A modern approach. 4th ed. (Global ed.) 2021: Pearson. [Kindle Edition]
  - (NJ) Neapolitan, R.E. and X. Jiang, Artificial intelligence : with an introduction to machine learning. 2nd edition. 2018, Boca Raton: CRC Press, Taylor & Francis Group.
  - (BL) Brachman, R.J. and H.J. Levesque, Knowledge representation and reasoning. 2004, Amsterdam ; Boston: Morgan Kaufmann. [Kindle Edition]

Appendix

Logical syntax and semantics

Propositional Logic Syntax

it has nothing to do with attaching meaning to them

- A formal language and semantics that give meaning to the well-formed sentences, which are called propositions.
- Syntax propositional logic only concerns with developing propositions
- Alphabet of propositional logic contains the following symbols:
  1. Letters of the English alphabet; that is, $A, B, C, \ldots, Z$, and each of these letters with an index (e.g., A_4).
  2. Logical values $True$ and $False$.
  3. Special symbols:
     ¬ (NOT) ∧ (AND) ∨ (OR) ⇒ (IMPLIES) ⇔ (IF AND ONLY IF) () (GROUPING).
  - ¬ is a unary connective and the symbols ∧, ∨, ⇒, and ⇔ are binary connectives.

Syntactic Rules

- The rules for creating propositions are as follows:

1. All letters, all indexed letters, and the logical values $True$ and $False$ are propositions. They are called atomic sentences or propositions.
2. If A and B are propositions, then so are:
   $\neg A, A \wedge B, A \vee B, A \Rightarrow B, A \Leftrightarrow B, and\ (A)$.
   They are called complex sentences or compound propositions.
   - $\neg A$ (not) is called the negation of A
   - A literal is either an atomic sentence (a positive literal) or a negated literal
   - $A \wedge B$ (and) is called the conjunction of A and B, and
   - $A \vee B$ (or) is called the disjunction of A and B.
   - $A \Rightarrow B$ (implies) is called an implication with premise (or antecedent) A and conclusion (or consequent) B
   - Implications are also called rules or if-then statements
   - $A \Leftrightarrow B$ (if and only if) is called a biconditional

Propositional Logic Semantics

- Semantics of propositional logic
  - gives meaning to the propositions.
  - consists of semantic rules for assigning either the value True or False to every proposition, with respect to a model.
    - E.g., For proposition with 3 symbols – P, Q, R, there will be 2^3 possible models – one with each combination of True or False for each symbol.
  - Such an assignment is called the truth value of the proposition.
  - If a proposition has truth value True, it is true; otherwise, it is false.

First Order Logic Syntactic Rules

- A term is one of the following:
  - A constant symbol
  - A variable symbol
  - A function symbol followed by one or more terms separated by commas and enclosed in parentheses
- An atomic formula is one of the following:
  - A predicate symbol
  - A predicate symbol followed by one or more terms separated by commas and enclosed in parentheses
  - Two terms separated by the = symbol
- A formula is one of the following:
  - An atomic formula
  - Operator ¬ followed by a formula
  - Two formulas separated by ∨, ∧, ⇒, or ⇔
  - A quantifier, a variable followed by a formula
- A sentence is a formula with no free variables
  - A free variable is not quantified by ∀ or ∃ symbol; otherwise it is bound.
- Examples:
  - $\forall x\ love(x, y)$ - Not a sentence – free variable y
  - $\forall x\ tall(x)$ – A sentence – no free variables

Source: NJ 3.1.1

Normal Forms and Special Clauses

- Basic definitions
  - A literal is a sentence of the form α or $\neg\alpha$, where α is an atomic sentence other than True or False.
  - A conjunctive clause is a conjunction of literals
  - A disjunctive clause (or clause) is a disjunction of literals.
- Normal forms
  - A sentence is in disjunctive normal form (DNF) if it is the disjunction of conjunctive clauses
  - A sentence is in conjunctive normal form (CNF) if it is the conjunction of disjunctive clauses.
  - In normal forms, literals can contain variables, which are assumed to be universally quantified
- Horn clause
  - A disjunction of literals of which at most one is positive
- Definite clause
  - A disjunction of literals of which exactly one is positive.
  - All definite clauses are Horn clauses
- Goal clause
  - A clause with no positive literals

First Order Logic Semantics

- A Model in FOL has the following components:

1. A nonempty set D of entities called a domain of discourse.
2. An interpretation I, which consists of the following:
   - Every entity in D is assigned to a constant symbol.
   - For each function, an entity is assigned to each possible input of entities to function.
   - Predicate '$True$' is always assigned value $True$, and predicate '$False$' value $False$.
   - For every other predicate or relation, $True$ or $False$ is assigned to each possible input of entities to the predicate.

- A sentence (or formula) from the first-order language is said to be $True$ in a model if it evaluates to true under the interpretation given by that model.

Source: NJ 3.1.2

Additional Resources

- Jupyter notebook on logic from RN reference book:
  - Install Anaconda on your computer to run Jupyter notebook: https://docs.anaconda.com/anaconda/install/
  - Download aima-python-master.zip directly from https://github.com/aimacode/aima-python/tree/master
  - **(PLEASE SCAN FOR VIRUSES BEFORE INSTALLATION!)**
  - Jupyter notebook file on logic is logic.ipynb and python implementation of the code is in logic4e.py